

Raport Proiect IS - Etapa 1

Nume: Iordache Ioana-Diana

Grupa: 344C1

Cerinta 1: Clasa personalizata in PyTorch

Fisierul CSV original contine datele in format long - fiecare imagine are 6 randuri (cate unul per tip de hemoragie), ceea ce complica accesul rapid la toate etichetele unei imagini in metoda `__getitem__`.

Am implementat o functie de preprocesare (`process_df`) care transforma CSV-ul in format wide (pivot), obtinand un singur rand per imagine cu toate cele 6 etichete ca si coloane separate. aceasta operatie, executata o singura data la inceput, permite acces constant $O(1)$ la toate etichetele unei imagini.

Lazy loading → clasa stocheaza in constructor doar metadatele (DataFrame-ul cu `slice_id` si etichete) si calea catre directorul cu imagini. Nicio imagine nu este incarcata in memorie la initializare. Imaginile sunt citite de pe disc individual, doar cand DataLoader apeleaza `__getitem__` pentru un anumit index.

Metoda `__len__`: returneaza numarul de randuri din DataFrame-ul preprocesat, reprezentand numarul total de imagini. aceasta metoda este necesara pentru ca DataLoader sa calculeze numarul de batch-uri si sa implementeze shuffling-ul corect.

Metoda `__getitem__`: reprezinta nucleul functionalitatii lazy loading. Pentru un index dat:

- extrage metadatele imaginii respective din DataFrame (`slice_id` si etichete)
- construiesc calea completa catre fisierul imagine (format: `id_{slice_id}_frame0.png`)
- incarca imaginea de pe disc in acest moment folosind PiL
- aplica transformari specifice (resize, augmentari, normalizare)
- converteste etichetele in tensor Pytorch si le returneaza impreuna cu imaginea procesata

Conversia la grayscale → toate imaginile sunt fortate la un singur canal prin `convert("L")`. Desi imaginile Ct sunt intrinsec grayscale, unele PNG-uri pot fi salvate gresit ca RGB (3 canale identice), ceea ce consuma memorie inutila. aceasta conversie asigura consistenta — toate imaginile au shape (1, H, W) dupa aplicarea `to_tensor()`.

Conversie tip date etichete → am intampinat o problema unde Pandas `pivot_table` producea coloane cu dtype `object` in loc de numeric, chiar pentru valori 0/1. Incercarea de a crea tensori Pytorch direct din astfel de array-uri genera erori de tip `RuntimeError: Could not infer dtype of numpy.object_`. solutia pe care am adoptat-o este conversia explicita la float prin `astype(float)` inainte de transformarea in numpy array. Acest pas asigura compatibilitate cu `BCEWithLogitsLoss`, care necesita tensori `float32`.

Clasa `RSNADataset` functioneaza mana in mana cu **`torch.utils.data.DataLoader`**.

Beneficiile lazy loading-ului devin evidente in acest context:

- ★ **Paralelizare:** setarea `num_workers=4` creeaza 4 procese care executa `__getitem__` in paralel, accelerand incarcarea cu un factor de $\sim 4\times$
- ★ **Batch processing:** pentru un batch de 32 imagini, `DataLoader` apeleaza `__getitem__` de 32 de ori (cate o data per imagine), distribuind apelurile între workeri
- ★ **Memory efficiency:** consumul de RAM ramane constant (per batch de 32 imagini) indiferent de dimensiunea dataset-ului

Cerinta 2: Impartire dataset pentru antrenare si validare

Am utilizat functia `train_test_split` din `sklearn.model_selection` pentru a imparti `DataFrame`-ul `train_clean` (preprocesat la Cerinta 1) in doua subseturi.

Parametrii alesi au fost:

- `test_size=0.2` - aloca 20% din date pentru validare, rezultand un split 80/20. Acest raport este standard in machine learning, oferind suficiente date pentru antrenare (80%) mentinand un set de validare reprezentativ (20%).
- `random_state=42` - setarea unui seed fix asigura **reproductibilitatea** rezultatelor. Rulari multiple ale codului vor produce aceleasi seturi train/validation, esential pentru debugging si compararea experimentelor.
- `shuffle=True` - amesteca randurile inainte de impartire. Fara shuffle, daca datele originale sunt ordonate (ex: toate imaginile cu hemoragii la inceput, cele sanatoase la final), split-ul ar produce seturi complet dezechilibrate. Shuffle-ul asigura ca ambele subseturi au distributii similare de clase.

Rezultat:

Train: 32282 imagini (80%), Validation: 8071 imagini (20%)

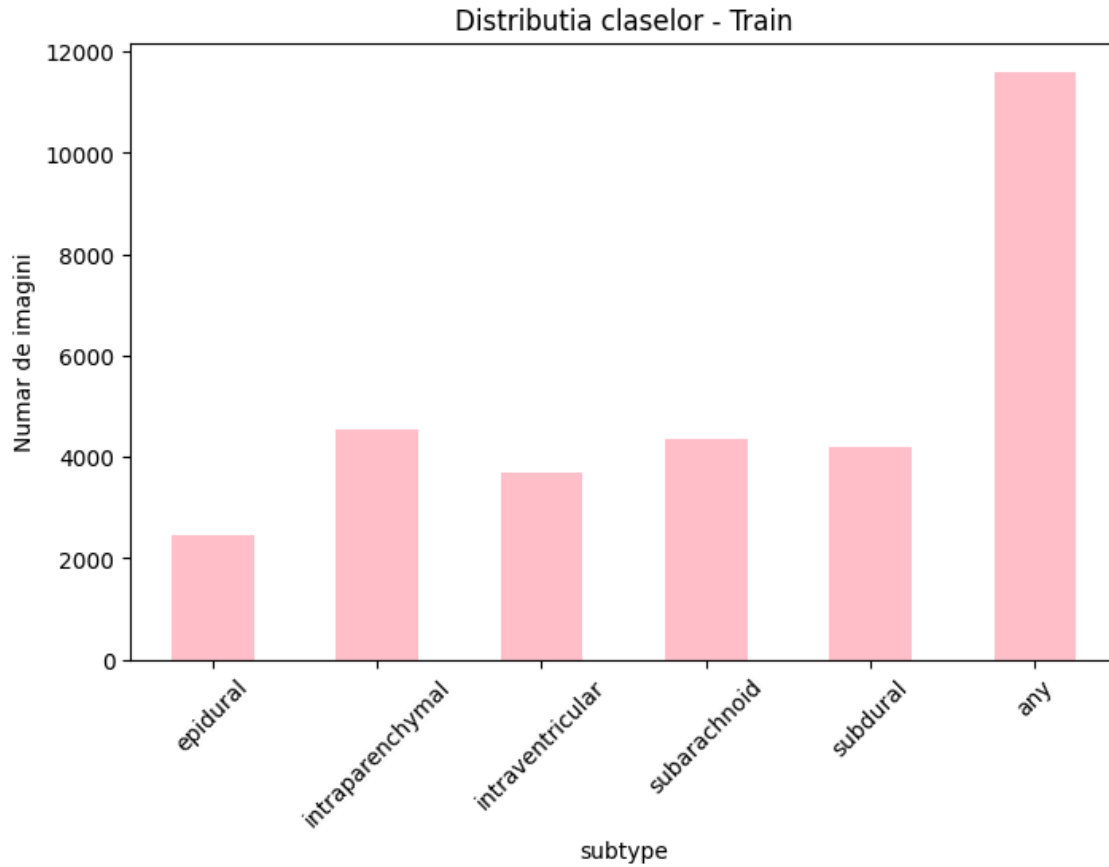
Implementarea actuala este compatibila direct cu strategii de cross-validation. Arhitectura bazata pe `DataFrame`-uri permite inlocuirea simpla a lui `train_test_split` cu `KFold` sau `StratifiedKFold` fara modificari ale clasei `RSNADataset` sau a training loop-ului.

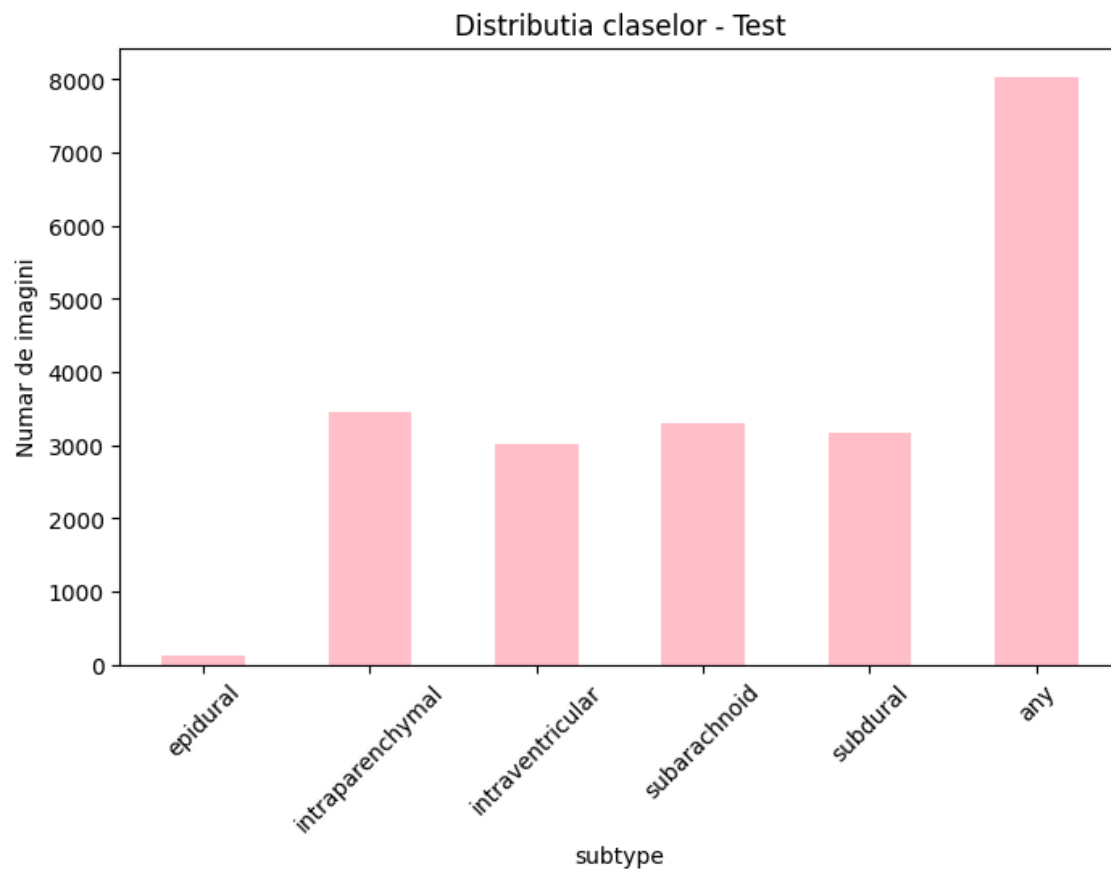
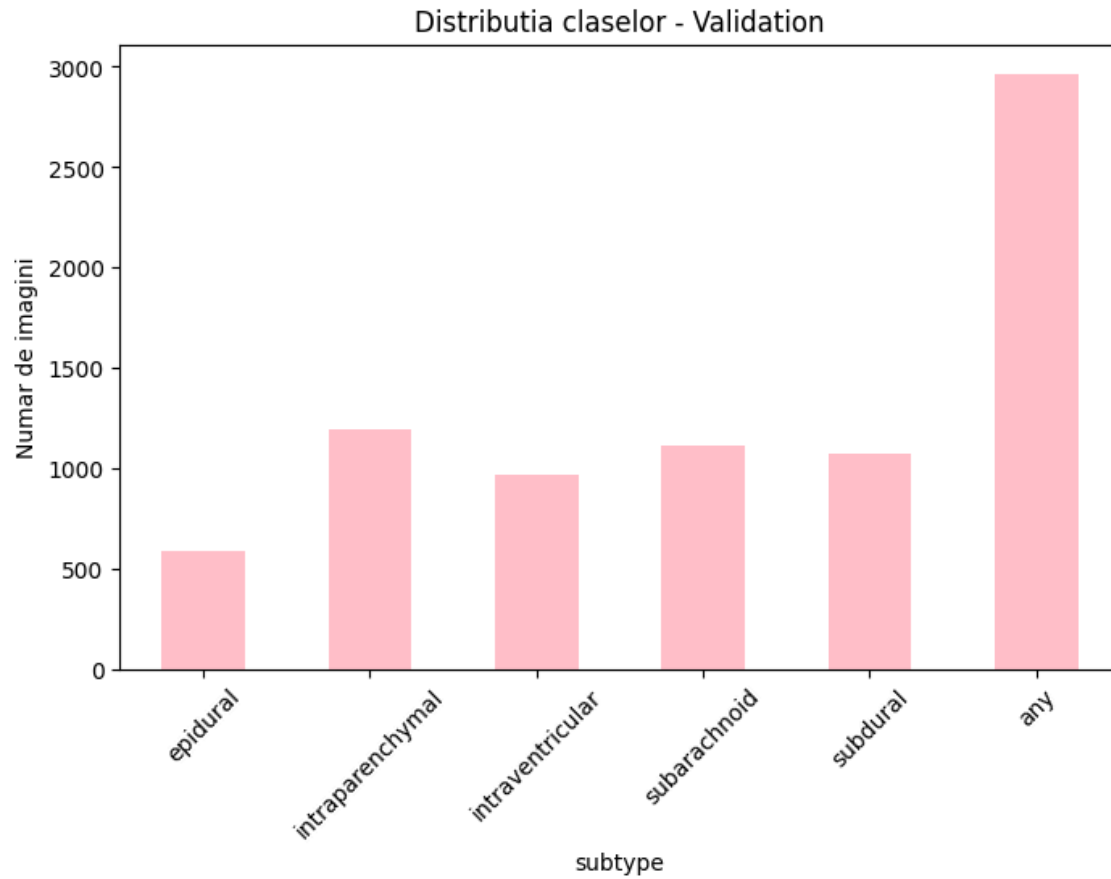
De ce nu K-Fold?

- K-Fold cross-validation e mai robust (antreneaza pe K configuratii diferite de train/val si medieaza rezultatele), dar e mult mai costisitor - inseamna sa antrenez modelul de K ori (de ex. 5 fold = 5 antrenari complete).
- Pentru baseline-ul de la etapa 1, am preferat sa experimentez rapid cu un singur split 80/20 si sa verific ca pipeline-ul functioneaza conform asteptarilor.
- Voi reveni cu K-Fold pentru a valida robustetea modelului si a obtine metrici mai fiabile in etapa urmatoare de tuning.

Cerinta 3: Vizualizare si analiza distributiei claselor

Am implementat o functie pentru vizualizarea distributiei claselor pe toate cele 3 seturi de date (train, validation, test). Functia calculeaza numarul de imagini pozitive pentru fiecare dintre cele 6 clase (epidural, intraparenchymal, intraventricular, subarachnoid, subdural, any) si le afiseaza intr-un grafic de bare.





Analiza grafica a relevat un dezechilibru intre clase pe toate cele 3 seturi:

TRAIN (total: 32282 imagini):

- any: ~11500 imagini (35.6%) - clasa dominanta pentru prezenta hemoragie
- intraparenchymal: ~4600 (14.3%)
- subarachnoid: ~4300 (13.3%)
- intraventricular: ~3700 (11.5%)
- subdural: ~4200 (13.0%)
- **epidural: ~2500 (7.7%) - CEA MAI RARA CLASA**

VALIDATION (total: 8071 imagini):

- **Distributie aproape identica cu train (proportii similare)!**
- any: ~3000 (37.2%)
- epidural: ~600 (7.4%)
- Celelalte clase: ~1000-1200 fiecare (12-15%)

TEST (total: 8899 imagini):

- any: ~8000 (89.9%) - MULT MAI FRECVENTA decat in train/val
- intraparenchymal: ~3500 (39.3%)
- subarachnoid: ~3400 (38.2%)
- intraventricular: ~3000 (33.7%)
- subdural: ~3200 (36.0%)
- epidural: ~120 (1.3%) - EXTREM DE RARA

Clasele intraparenchymal, intraventricular, subarachnoid si subdural au frecvente relativ apropiate (11-15% pe train/val, 33-39% pe test), ceea ce nu reprezinta un dezechilibru extrem. Clasa any este dominanta pe toate seturile (35-90%), dar acest lucru este asteptat - any=1 inseamna prezenta oricarui tip de hemoragie.

PROBLEMA MAJORA: EPIDURAL

Epidural hemorrhage este dramatic sub-reprezentata:

- Train: 2500 imagini (7.7%)
- Validation: 600 imagini (7.4%)
- Test: 120 imagini (1.3%)

Am evaluat si implementat mai multe strategii:

1. WeightedRandomSampler (IMPLEMENTAT - solutia principala)

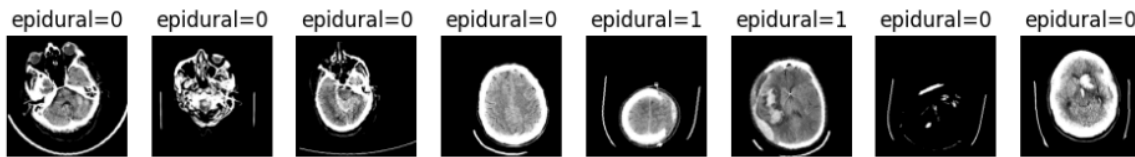
Principiu: Supraesantioneaza imaginile cu epidural in timpul antrenarii. Fiecare imagine cu epidural=1 are probabilitate mai mare de a fi selectata in batch-uri.

Implementare testata: Am experimentat cu mai multe weight-uri ($w=2$, $w=4$, $w=6$) pentru epidural.

Configuratie finala: $w=4$ (imaginile cu epidural au probabilitate de selectie de $4\times$ fata de celelalte)

Effect: In batch-urile de training, epidural apare in ~20-25% din imagini (vs 7.7% natural), crescand semnificativ expunerea modelului la aceasta clasa rara.

Batch sampled (augmented pentru epidural)



Fracție epidural în batch: 0.25

Avantaje:

- Simplu de implementat (doar modificare DataLoader cu parametru sampler)
- Modelul vede epidural mai des, invata mai bine sa detecteze pattern-urile specifice
- Recall crescut pe epidural (detecteaza mai multe cazuri reale)

Dezavantaje:

- Modelul poate invata sa prezica epidural prea des, rezultand false positives pe test
- Necesita mai multe epoci pentru convergenta (modelul vede exemple duplicate)

Rezultate obtinute (w=4, 18 epoci cu early stopping):

- Epidural F1 test: 0.221 (precision=0.134, recall=0.634)
- Comparatie fara sampler (20 epoci): F1=0.158 (precision=0.090, recall=0.667)
- Imbunatatire: +40% F1, dar precision ramane slaba fata de restul claselor (multe false positives)

2. Class weights in loss function (IMPLEMENTAT - complementar cu sampler)

Principiu: BCEWithLogitsLoss cu parametrul `pos_weight` calculat automat din distributia train. Loss-ul penalizeaza mai mult false negatives pe clase rare (un epidural ratat costa mai mult in loss decat un intraparenchymal ratat).

Implementare: `pos_weight = neg_count / pos_count` per clasa

Pentru epidural: `pos_weight ≈ 5.2` (29782 negative / 2500 positive in train)

Avantaje:

- Nu modifica distributia batch-urilor (complementar cu sampler)
- Gradientul este mai mare pentru erorile pe clase rare, fortand modelul sa acorde mai multa atentie
- Se combina natural cu `WeightedRandomSampler`

Dezavantaje:

- Efect mai subtil decat sampler (nu garanteaza ca modelul "vede" destule exemple)
- Necesita tuning fin (`pos_weight` prea mare poate cauza instabilitate in antrenare)

3. Threshold tuning per clasa (IMPLEMENTAT - post-processing)

Principiu: In loc de threshold fix 0.5 pentru toate clasele, optimizez threshold-ul individual pe validation set pentru a maximiza F1 per clasa.

Implementare: Grid search pe praguri 0.1-0.9 (pas 0.05) pentru fiecare clasa, alegand threshold-ul care maximizeaza F1 pe validation.

Rezultate:

```
Optimizing threshold per clasa (validation set):
epidural      : threshold=0.70, val_f1=0.647
intraparenchymal: threshold=0.75, val_f1=0.738
intraventricular: threshold=0.85, val_f1=0.809
subarachnoid   : threshold=0.90, val_f1=0.601
subdural       : threshold=0.65, val_f1=0.572
any            : threshold=0.55, val_f1=0.860
```

Avantaje:

- Post-processing simplu (nu necesita reantrenare)
- Compenseaza partial bias-ul creat de sampler (threshold mai mare filtreaza false positives)
- Imbunatateste F1 pe validation

Dezavantaje:

- Threshold-urile sunt calibrate pe validation (care are distributie 7.4% epidural)
- Pe test (1.3% epidural), threshold 0.75 e posibil inca prea agresiv (multe false positives)

```
--- Metrics for 'epidural' ---
TN (True Negative):  8271 | FP (False Positive):  505
FN (False Negative):   45 | TP (True Positive):   78
Precision: 0.134 | Recall: 0.634 | F1-score: 0.221
```

positives)

4. Solutii netestate (de analizat daca sunt mai potrivite pentru Etapa 2)

Focal Loss: Inlocuieste `BCEWithLogitsLoss` cu Focal Loss, care penalizeaza mai agresiv exemple greu clasificabile (reduce false positives pe clase rare mai eficient decat `pos_weight` simplu).

SMOTE imagistic / Mixup: Genereaza exemple sintetice de epidural prin interpolare intre imagini existente (ex: mixup intre 2 imagini cu epidural), mai realistic decat simpla duplicare din sampler (mi s-a parut relevanta aceasta lucrare:

https://aipublications.com/uploads/issue_files/2IJECC-SEP20245-Anomaly.pdf)

Decizia finala pentru baseline

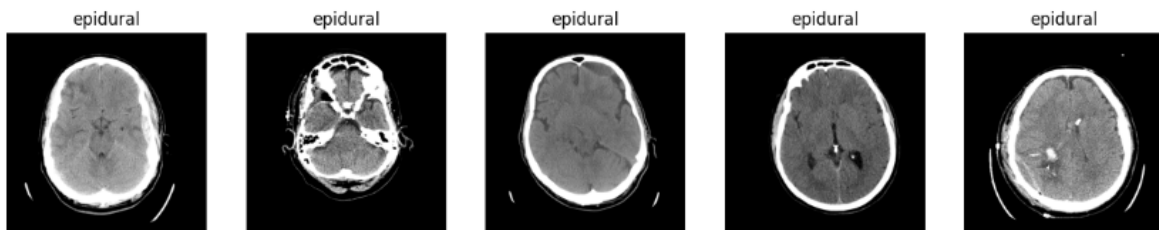
Am ales combinatia `WeightedRandomSampler (w=4) + pos_weight` in loss + **threshold tuning** per clasa.

- ★ Oferă cel mai bun compromis pentru baseline (epidural $F1=0.221$ pe test vs 0.158 fara sampler)
- ★ Testele comparative ($w=2$, $w=4$, $w=6$, fara sampler) au confirmat ca $w=4$ maximizeaza $F1$ pe epidural fara a degrada semnificativ celelalte clase
- ★ Celelalte 5 clase (intraparenchymal, intraventricular, subarachnoid, subdural, any) generalizeaza foarte bine pe test

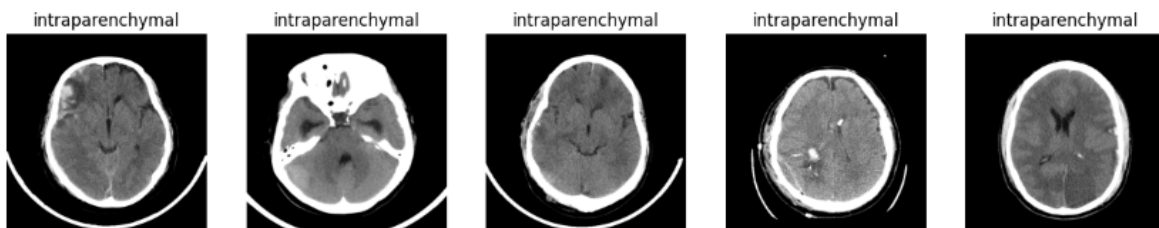
Cerinta 4: Analiza vizuala detaliata a imaginilor

Am extras si afisat cate 5 imagini din fiecare clasa (epidural, intraparenchymal, intraventricular, subarachnoid, subdural, any) pentru a observa caracteristicile vizuale si variabilitatea intra-clasa.

Epidural: Variabilitate mare de marime (de la mici acumulari la expansiuni semnificative) si locatie (temporal, frontal, parietal). Forma tipica biconvexa (lentila) la periferie.



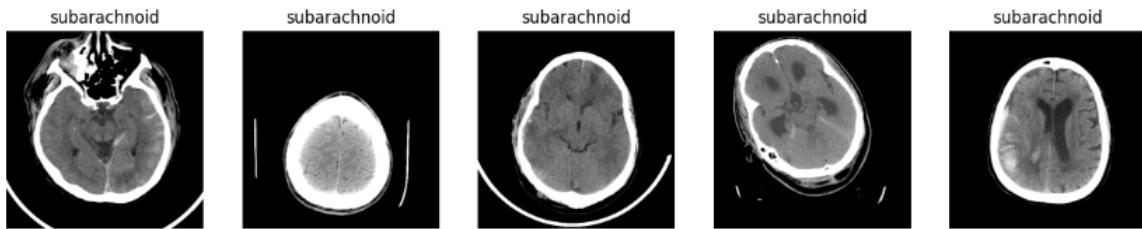
Intraparenchymal: Variabilitate maxima - localizare in orice zona a tesutului cerebral, marime de la mici petehii la hematoame mari. Aspect: zone hiperdense rotunjite/neregulate in interiorul tesutului.



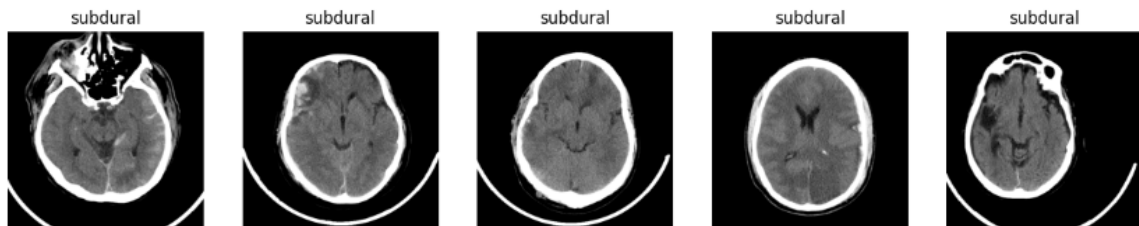
Intraventricular: Variabilitate mica - localizare constanta in ventriculii cerebrali (cavitati centrale). Forma ventriculilor este anatomic fixa, facilitand recunoasterea.



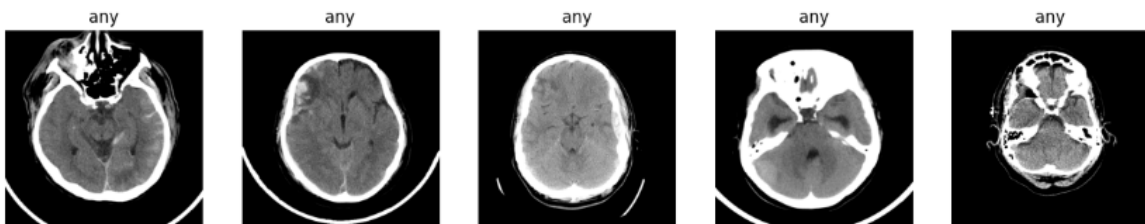
Subarachnoid: Variabilitate mare - de la hemoragii mici localizate in fisuri corticale la hemoragii difuze. Aspect: hiperdensitati liniare/ramificate la suprafata creierului.



Subdural: Variabilitate moderata de marime si extindere. Forma tipica crescativa (semiluna) la periferie, urmareste conturul emisferei.



Any: Variabilitate maxima (orice combinatie a claselor anterioare, de la leziuni simple la multiple complexe).



Similaritati intre clase (dificultati asteptate):

- **Epidural vs Subdural (problema majora):** Ambele apar la periferia creierului ca zone hiperdense. Diferenta cheie (forma biconvexa vs semiluna) este subtila pe rezolutie $256 \times 256 \Rightarrow$ risc major de confuzie pentru model.
- **Intraparenchymal vs localizari atipice ale altor clase:** Hemoragii intraparenchymale mari pot comprima ventriculii sau extinde catre spatiul subarahnoidian, creand aspect mixt.
- **Subarachnoid vs artefacte:** Hemoragii subarahnoidiene mici pot fi confundate cu variatii normale de contrast sau artefacte de imagistica.

Diferente clare intre clase (facilitati pentru model):

- **Intraventricular vs restul:** Localizare anatomica unica (ventriculii au forma si pozitie constanta) → cel mai usor de clasificat.
- **Intraparenchymal vs epidural/subdural:** Localizare interna vs periferica → diferentiere clara.

- **Any vs imagini sanatoase:** Task binar simplu (prezenta vs absenta hiperdensitati anormale) → performanta asteptata excelenta.

Cerinta 5: Verificarea consistentei datasetului

Am analizat toate cele 40353 imagini din train set, extragand pentru fiecare: dimensiuni (height, width), numar canale si range pixeli (min, max).

NUMAR CANALE:

- Toate imaginile au 1 canal (grayscale)
- Nu exista inconsistente

DIMENSIUNI:

- 40341 imagini (99.97%): 512×512 pixels
- 12 imagini (0.03%): **dimensiuni variabile** (408×374, 430×404, 464×464, 638×490, 666×512)

RANGE PIXELI:

- Toate imaginile: [0, 255] (uint8 standard)
- Unele imagini au contrast redus (pixel_max < 255)

Solutii implementate

PROBLEMA: Dimensiuni neuniforme (12 imagini diferite)

SOLUTIE: `transforms.Resize(256, 256)` in toate pipeline-urile

EFFECT: Toate imaginile ajung 256×256 in model (necesar pentru batching in DataLoader)

PROBLEMA: Range pixeli variabil intre imagini

SOLUTIE: `transforms.Normalize(mean=[0.5], std=[0.5])` dupa `ToTensor()`

EFFECT: Scalare uniforma la [-1, 1], distributie consistenta pentru antrenare

MASURA PREVENTIVA: `.convert("L")` fortat in `RSNADataset` (asigura 1 canal chiar daca exista PNG-uri salvate gresit ca RGB)

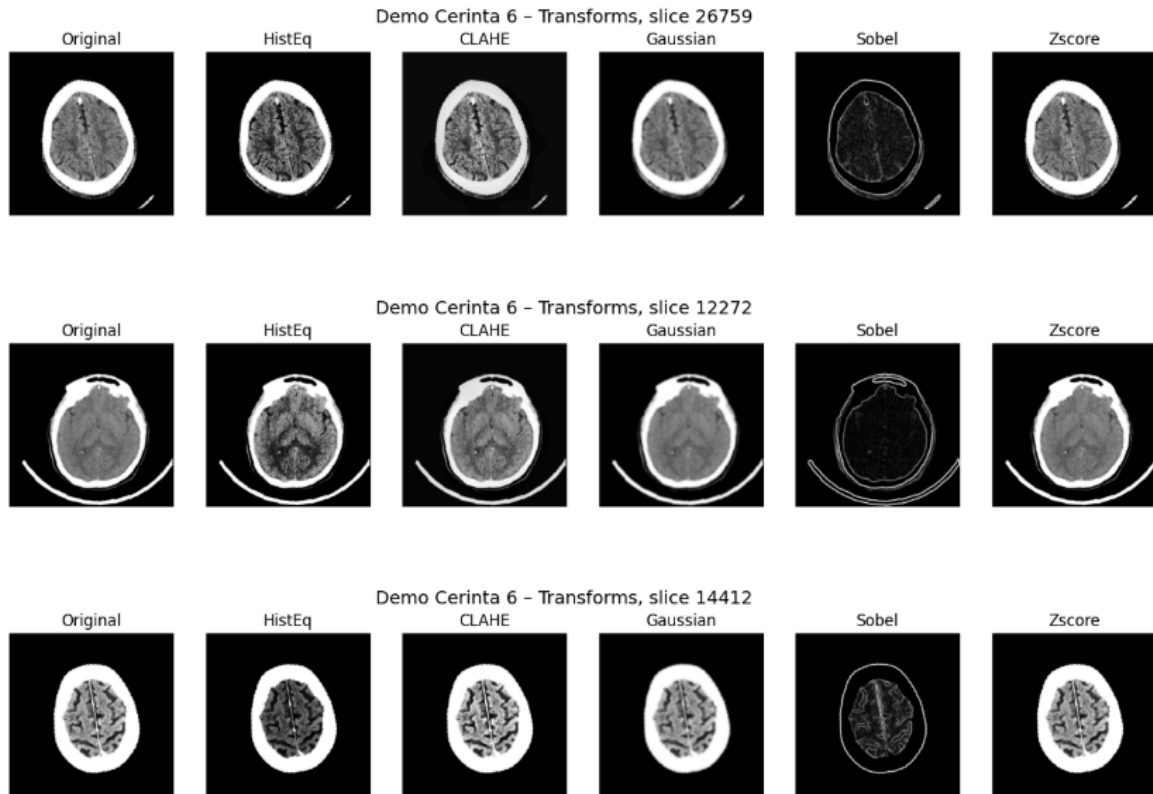
Verificare finala

Sample post-transformari:

- Shape: `torch.Size([1, 256, 256])` - corect
- Range: [-1.0, 1.0] - corect

Cerinta 6: Operatii de preprocesare si normalizare

Am implementat 5 transformari custom si le-am aplicat pe 5 imagini random din train set pentru a compara vizual impactul fiecarei tehnici. Pentru fiecare imagine am afisat versiunea originala alaturi de cele 5 variante transformate, permitand o evaluare directa a efectelor.



(exemplu pe 3 dintre cele 5 imagini selectate)

Histogram Equalization

Din imaginile afisate observ ca HistEq produce o **usoara crestere a contrastului**, dar efectul e foarte subtil pe aceste CT-uri. Pe a doua imagine (slice 12272) se observa o amplificare foarte usoara a structurilor interne, dar fara efecte dramatice. Interesant este ca HistEq nu e atat de agresiv pe cat ma asteptam - probabil pentru ca aceste imagini CT sunt deja normalizate clinic. Totusi, nu aduce imbunatatiri semnificative vizibile, asa ca **nu justifica includerea in pipeline**.

CLAHE (Contrast Limited Adaptive Histogram Equalization)

CLAHE produce efecte similare cu HistEq pe aceste exemple specifice - **diferentele sunt minimale vizual**. Pe slice 26759 si 14412 ambele tehnici (HistEq si CLAHE) arata aproape identic cu originalul. Asta sugereaza ca imaginile au deja un contrast bun si nu necesita ajustari majore. Totusi, **CLAHE ramane in pipeline pentru ca pe imagini cu contrast mai slab (care exista in dataset dar nu sunt in aceste 3 exemple) efectul adaptiv local va fi mai vizibil si util**. Limitarea de amplificare din CLAHE **previne artefactele** pe care HistEq le-ar putea introduce pe imagini cu zgomot mai mare.

Gaussian Blur

Efectul de blur e **vizibil dar subtil** - marginile devin usor mai **netede**, **structurile fine** (fisuri, vase mici) **pierd claritate**. Pe slice 12272 se observa ca conturul exterior al craniului

devine usor difuz. Pe slice 14412 detaliile fine din substanta alba devin mai putin distincte. Pentru clasificarea hemoragiilor, unde marginile clare sunt importante (mai ales pentru diferentierea epidural vs subdural), aceasta pierdere de claritate e nedorita. **Decizia de a exclude Gaussian Blur din pipeline ramane justificata.**

Sobel Edge Detection

Sobel produce imagini cu niste **contururi foarte clare**: craniul, structurile ventriculare, fisurile corticale sunt perfect delimitate. Pe slice 12272 se vad clar limitele ventriculilor si conturul hemoragiei (zona mai densa). Insa toata **informatia de intensitate dispare** - nu mai putem diferentia tesut normal de hemoragie bazandu-ne doar pe intensitatea pixelilor, ci doar pe prezenta unei margini. Pentru clasificare multi-label unde hiperdensitatea este un marker cheie al hemoragiei, aceasta pierdere de informatie e critica. **Sobel ramane exclus din pipeline.**

Z-score Normalization

Vizual, diferenta fata de original e **imperceptibila** pe aceste exemple - imaginile arata practic identic. Asta e exact ce trebuie sa se intample, pentru ca normalizarea e doar o rescalare liniara care nu ar trebui sa schimbe aspectul vizual. Valoarea ei sta in consistenta numerica pentru antrenare (toate imaginile ajung pe aceeasi scala), nu in imbunatatirea vizuala. **Nu l-am inclus asadar nici pe acesta in pipeline.**

Augmentari suplimentare

Pe langa **CLAHE** si **Normalize**, am adaugat in `train_transform` **augmentari geometrice care au sens pe datele medicale analizate**: `RandomHorizontalFlip` (creierul e aproximativ simetric orizontal), `RandomRotation( $\pm 10^\circ$ )` pentru variatii de pozitionare a pacientului, si `ColorJitter` pentru robustețe la diferente de calibrare intre scanere. **Nu am inclus vertical flip** (creierul nu e simetric vertical), **crop agresiv** (risc sa tai hemoragii la margini), sau **deformari elastice** (denatureaza forma leziunilor).

Cerinta 7: Proiectare si antrenare model CNN

Alegerea arhitecturii

Am ales **ResNet18** ca arhitectura de baseline pentru acest task de clasificare multi-label. Decizia s-a bazat pe mai multe considerente practice si teoretice:

- ★ **Skip connections**: ResNet18 rezolva problema vanishing gradient prin skip connections care permit gradientilor sa se propage direct prin retea, facilitand antrenarea retelelor mai profunde. Pentru imagini medicale unde pattern-urile pot fi subtile si complexe (diferenta geometrica epidural vs subdural), o retea mai profunda poate invata mai bine sa diferentieze intre tipurile de hemoragii.
- ★ **Arhitectura standard in medical imaging**: ResNet18 e folosita ca baseline in majoritatea studiilor pe imagini CT si radiografii (referinta frecventa in literatura

RSNA, papers pe ChexPert, etc.). Asta ofera un punct de comparatie valid si asigura ca arhitectura e potrivita pentru task-ul medical.

- ★ **Compromis profunzime vs viteza:** Cu 18 layere, e suficient de profunda pentru pattern-uri complexe, dar suficient de compacta pentru antrenare rapida (cam 2.5 minute per epoca pe GPU Kaggle).

```
class ResNet18ICH(nn.Module):
    def __init__(self, num_classes=6):
        # Conv initial: 1 canal input (grayscale CT) in loc de 3 (RGB)
        self.conv1 = nn.Conv2d(1, 64, kernel_size=7, stride=2, padding=3, bias=False)

        # 4 layere reziduale cu BasicBlock (64->128->256->512 filtre)
        self.layer1 = self._make_layer(64, 2, stride=1)
        self.layer2 = self._make_layer(128, 2, stride=2)
        self.layer3 = self._make_layer(256, 2, stride=2)
        self.layer4 = self._make_layer(512, 2, stride=2)

        # 2 head-uri separate la output
        self.head_five = nn.Linear(512, 5) # epidural, intraparenchymal, ...
        self.head_any = nn.Linear(512, 1) # any (orice hemoragie)
```

! Am decis sa folosesc 2 head-uri separate pentru a permite modelului sa invete ca *any* e corelat dar nu identic cu celelalte 5 clase.

DataLoader-ele folosesc:

- batch_size=32 (balans intre viteza si memorie GPU)
- num_workers=4 (paralelizare incarcare date)
- WeightedRandomSampler pentru train (w=4 pentru epidural, supraesantioneaza clasa rara)
- shuffle=False pentru validation/test (consistenta metrice)

Functia de pierdere si optimizator

Loss function: BCEWithLogitsLoss cu pos_weight

Am calculat automat pos_weight (raport negativ/pozitiv per clasa) iterand prin train_loader:

```
pos_weight = neg_counts / pos_counts
# Rezultat: [2.92, 5.51, 8.37, 5.88, 5.83, 1.06]
```

BCEWithLogitsLoss penalizeaza mai tare false negatives pe clase rare. Un epidural ratat (pos_weight=2.92) costa de ~3× mai mult in loss decat un "any" ratat (pos_weight=1.06).

Optimizer: Adam cu learning rate 1e-4

Adam e optimizer **standard** pentru baseline-uri CNN (adaptive learning rate, momentum).
 $LR=1e-4$ e valoare conservatoare pentru start, evita divergenta pe primele epoci.

Scheduler: ReduceLROnPlateau

Reduce LR cu factor 0.5 daca `val_loss` nu scade 2 epoci consecutive. Am observat reduceri la epoca 12 ($1e-4 \rightarrow 5e-5$) si epoca 16 ($5e-5 \rightarrow 2.5e-5$), ajutand convergenta fina.

Antrenarea modelului

Setup training:

- 20 epoci maxime
- **Early stopping** cu `patience=5` (opreste daca `val_loss` stagnează 5 epoci consecutive)
- **Salvare 2 modele:** best by `val_loss` si best by macro F1
- **Seed fix** (`random_state=42`) pentru reproductibilitate

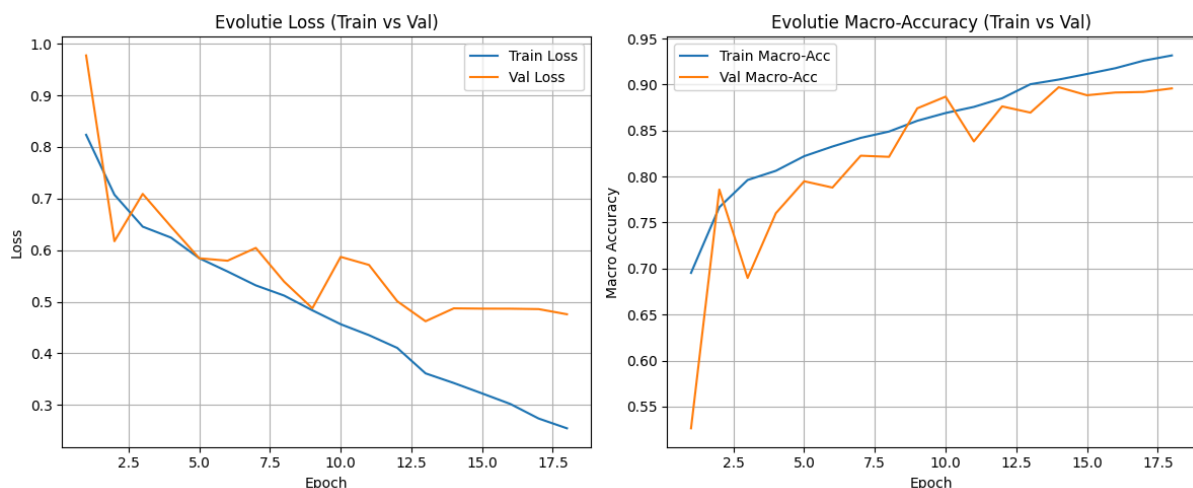
Metrici monitorizate per epoca:

- Train/Val loss (convergenta generala)
- Macro-accuracy (medie per-class accuracy, evita dominarea claselor frecvente)
- Exact-match accuracy (toate cele 6 etichete corecte simultan)
- Per-class precision/recall/F1 (identifica clase problematice)

Evolutie antrenare:

La epoca **18** early stopping s-a declansat (`val_loss` stagna de la epoca 13).

Modelul final: epoca 18 cu macro F1=0.693 pe validation.



Observatii din grafice

- **Loss:** Train loss scade lin si constant ($0.82 \rightarrow 0.25$), aratand invatare progresiva. **Val loss** scade treptat in epocile 1-9 ($0.98 \rightarrow 0.49$), apoi spre final se pastreaza pe la o valoare de ~ 0.46 .

- **Macro-accuracy:** Crește constant pe ambele seturi (train: 0.64 → 0.93, val: 0.53 → 0.90). Oscilări epoci 1-11 din cauza sampler-ului (distributie artificială epidural în batch-uri). După epoca 12 (când LR scade prima dată), avem o **stabilizare** clară.
- **Convergența:** Epoca 9 e *turning point* vizibil - val_loss face jump de la 0.59 la 0.49, moment când modelul “*înțelege*” task-ul multi-label.

Evaluarea modelului

Threshold tuning pe validation

Cu threshold fix 0.5, modelul **nu era calibrat optim**. Am implementat grid search (praguri 0.1-0.9, step 0.05) pe validation pentru a maximiza F1 per clasă:

Threshold-uri optime per clasă (calibrate pe validation):

Clasă	Threshold optimal
epidural	0.70
intraparenchymal	0.75
intraventricular	0.85
subarachnoid	0.90
subdural	0.65
any	0.55

Clasele rare (epidural, subdural) au praguri mai mici pentru recall mai mare, clasele moderate (intraventricular, subarachnoid) au praguri mai mari pentru reducere false positives.

Clasa any are threshold aproape de default (0.55) deoarece este foarte echilibrată (pos_weight=1.06) și modelul are deja precision excepțională (0.988). Nu necesită filtrare agresivă prin threshold ridicat, spre deosebire de clase cu multe false positives (ex: subarachnoid cu threshold=0.90 pentru a filtra 1331 FP).

Metrici finale pe test (cu threshold-uri optimizate)

Classification Report - Metrici per clasă pe test set:

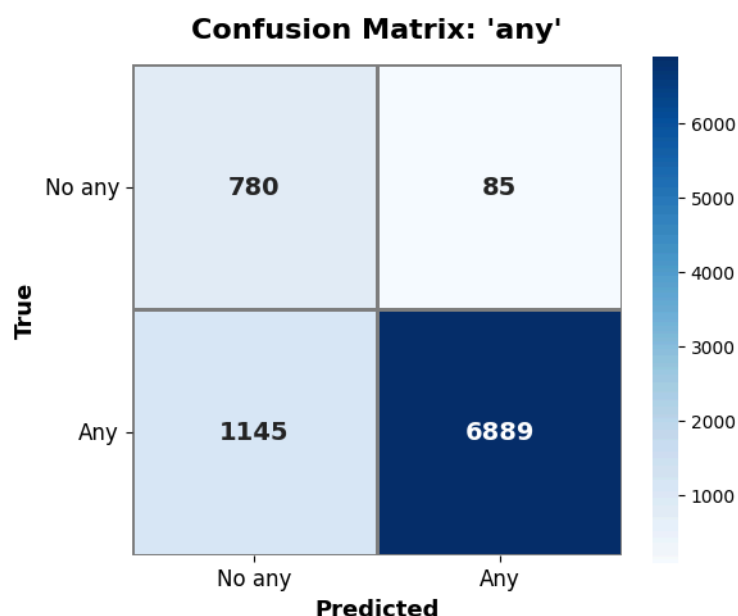
Clasă	Precision	Recall	F1-score	Support
epidural	0.134	0.634	0.221	123

intraparenchymal	0.731	0.743	0.737	3463
intraventricular	0.790	0.849	0.819	3011
subarachnoid	0.620	0.659	0.638	3292
subdural	0.650	0.610	0.629	3170
any	0.988	0.857	0.918	8034
macro avg	0.652	0.725	0.660	21093
weighted avg	0.804	0.768	0.783	21093

Exact-match accuracy test: 0.341 (34% din imagini au toate cele 6 etichete corecte simultan)

Per-class accuracy pe test set:

Clasa	Accuracy
epidural	0.938
intraparenchymal	0.794
intraventricular	0.873
subarachnoid	0.724
subdural	0.744
any	0.862



Analiza matricilor de confuzie

Modelul prezinta performante diferite pe cele 6 clase, reflectand atat dificultatea intrinseca a task-ului cat si caracteristicile fiecarei categorii de hemoragie:

Intraventricular si **any** sunt cele mai reusite clase. **Intraventricular** atinge **F1=0.819** cu un **balans excelent intre precision** (0.790) **si recall** (0.849). Localizarea anatomica unica a ventriculilor cerebrali (structuri cu forma si pozitie constanta) faciliteaza invatarea pattern-urilor specifice. **Any** obtine **rezultate aproape perfecte** (F1=0.918, precision=0.988), cu doar 85 false positives din 6974 predictii pozitive. Task-ul de detectie binara (prezenta vs absenta oricarui tip de hemoragie) este semnificativ mai simplu decat discriminarea fina intre cele 5 subtipuri.

Clasele **intraparenchymal**, **subarachnoid** si **subdural** au performante solide si consistente. **Intraparenchymal** obtine **F1=0.737** cu **balans bun precision-recall** (0.731/0.743), demonstrand generalizare corecta de la validation la test. **Subarachnoid** (F1=0.638) si **subdural** (F1=0.629) mentin performante decente in ciuda variabilitatii mari intra-clasa observate la Cerinta 4. Modelul reuseste sa diferentieze aceste tipuri de hemoragii cu acuratete acceptabila pentru un baseline.

Epidural prezinta provocari specifice legate de raritatea extrema pe test (1.3% din imagini vs 7.7% in train). Recall-ul de 0.634 arata ca modelul detecteaza corect majoritatea cazurilor reale de epidural (78 din 123 true positives identificati). Precision-ul mai scazut (0.134) reflecta dificultatea calibrarii pe o distributie atat de rara - cu doar 123 exemple pozitive pe test, orice decizie de threshold are impact major. Rezultatul este de asteptat in aceste conditii si reprezinta un punct de plecare solid pentru optimizari in Etapa 2 <3